

Proyecto 1

Predicción de Popularidad en Canciones de Spotify

Modelos Predictivos Basados en Árboles y Ensamblés

1. Descripción del Problema

El objetivo es predecir la popularidad de canciones en Spotify (escala 0–100), calculada en función del volumen y recencia de reproducciones. El dataset contiene 79.800 registros de entrenamiento y 34.200 de test, con variables de audio (danceability, energy, loudness, tempo, entre otras) y variables de identidad textual (track_name, artists, track_genre). La métrica de evaluación oficial es el RMSE.

2. Preprocesamiento y Análisis Exploratorio

2.1 División train / validación

Los datos se dividieron en 80% entrenamiento y 20% validación (random_state=42). Los originales se guardaron para poder alternar entre configuraciones de variables sin necesidad de recargar datos.

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42  
)  
  
X_train_orig = X_train.copy()  
X_test_orig = X_test.copy()
```

2.2 Exploración inicial

Se revisaron tipos de datos, cardinalidad, valores nulos y valores únicos por columna. No se encontraron nulos en variables numéricas. Las variables de texto (track_id, artists, album_name, track_name, track_genre) presentan cardinalidad extremadamente alta — desde miles hasta decenas de miles de valores únicos.

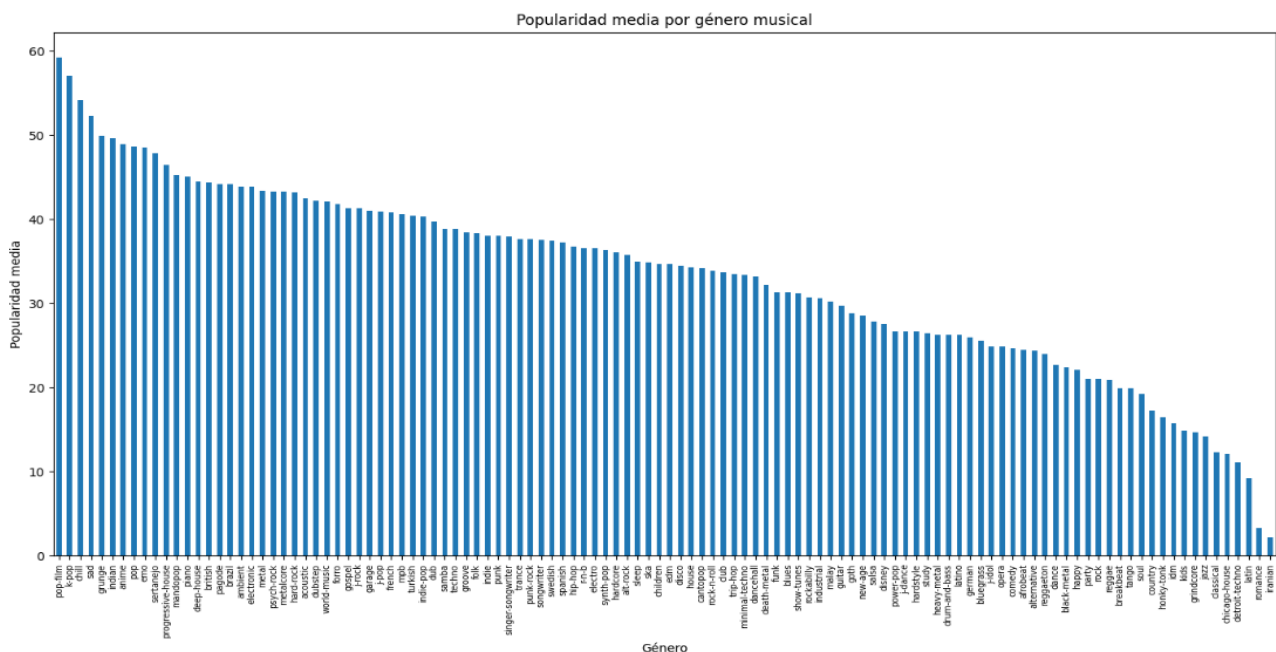
```
print(len(dataTraining))  
print(dataTraining.shape)  
print(len(dataTesting))  
print(dataTesting.shape)  
  
79800  
(79800, 21)  
34200  
(34200, 21)
```

```
# Valores Nulos  
tabla_nulos = pd.DataFrame({  
    "nulos": dataTraining.isnull().sum(),  
    "porcentaje": (dataTraining.isnull().mean() * 100).round(2)  
})  
  
# filtrar solo variables con nulos  
tabla_nulos = tabla_nulos[tabla_nulos["nulos"] > 0]
```

nulos	porcentaje
-------	------------

2.3 Variable track_genre y target encoding exploratorio

Con 114 géneros y diferencias de popularidad media de hasta 44 puntos entre géneros (15 a 59), track_genre es el factor más determinante del dataset. Para los modelos preliminares se aplicó target encoding calculado exclusivamente sobre el train (evitando data leakage):



```
# Para Incluir
genre_mean = dataTraining.groupby('track_genre')['popularity'].mean()
dataTraining['genre_encoded'] = dataTraining['track_genre'].map(genre_mean)
dataTesting['genre_encoded'] = dataTesting['track_genre'].map(genre_mean)
```

2.4 Duplicados por track_id

Se identificaron 9.826 canciones en más de un género. Con target encoding aplicado, cada aparición genera un valor distinto de genre_encoded, por lo que cada fila aporta información diferente y se conservaron todos los registros.

```
Canciones que aparecen en 1 género: 56894
Canciones que aparecen en 2+ géneros: 9826
Canciones que aparecen en 5+ géneros: 139
Máximo géneros para una canción: 7

Canción más repetida: 'Never Gonna Give You Up' de The Black Keys
Aparece en 7 géneros:
['alt-rock', 'punk-rock', 'punk', 'garage', 'blues', 'alternative', 'rock']

=== Ejemplo de canción en múltiples géneros ===
   track_name  artists track_genre  genre_encoded  popularity  danceability  energy
26786  Better  Pink Sweat$;Kirby    chill      54.165498         0         0.613  0.471
37758  Better  Pink Sweat$;Kirby    soul       19.195291         0         0.613  0.471
```

2.5 Canciones con popularity = 0

El 14.1% del dataset (11.230 registros) presenta popularity = 0. El análisis comparativo de features de audio no evidenció diferencias sustanciales respecto al resto, confirmando que son canciones válidas con escasa reproducción reciente. Se conservaron para no sesgar el modelo.

```
Total filas: 79800
Canciones con popular=0: 11230 (14.1%)

Top 10 géneros con más canciones en 0:
track_genre
iranian      475
romance     459
jazz        449
soul        441
latin       408
country     408
rock        346
alternative 343
dance       341
reggae      281
Name: count, dtype: int64

Comparación de features - popularity=0 vs resto:
           popular=0  popular>0
danceability    0.575    0.566
energy          0.613    0.646
loudness       -8.246   -8.267
acousticness    0.343    0.310
instrumentalness 0.105    0.166
```

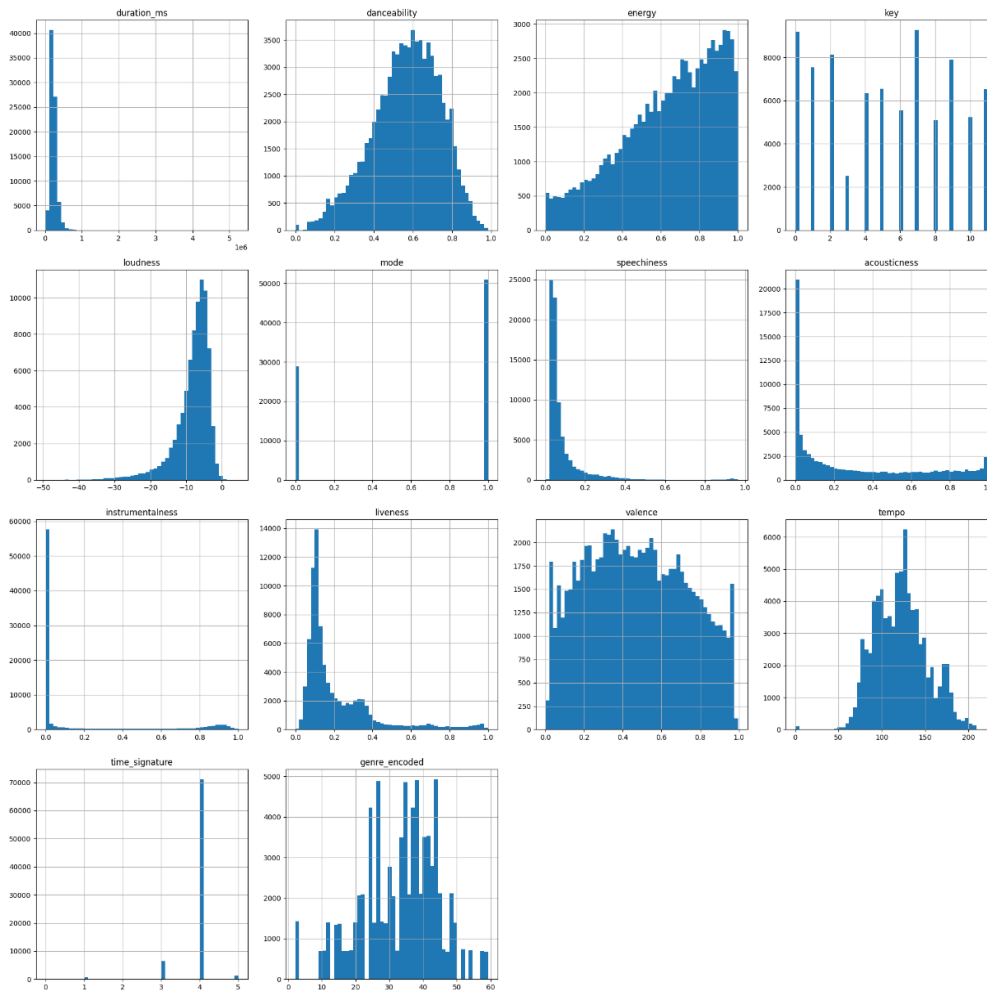
2.6 Análisis de outliers

El análisis IQR identificó outliers pronunciados en instrumentalness (22.1%), speechiness (11.6%) y time_signature (10.8%). Dado que los modelos basados en árboles son robustos a valores extremos, no se aplicó tratamiento de outliers.

	Q1	Q3	IQR	lim_inf	lim_sup	n_outliers	pct_outliers
instrumentalness	0.000000	0.0505	0.050500	-0.075750	0.126250	17645	22.11
speechiness	0.035900	0.0845	0.048600	-0.037000	0.157400	9277	11.63
time_signature	4.000000	4.0000	0.000000	4.000000	4.000000	8598	10.77
liveness	0.097900	0.2730	0.175100	-0.164750	0.535650	6032	7.56
loudness	-10.006000	-5.0000	5.006000	-17.515000	2.509000	4351	5.45

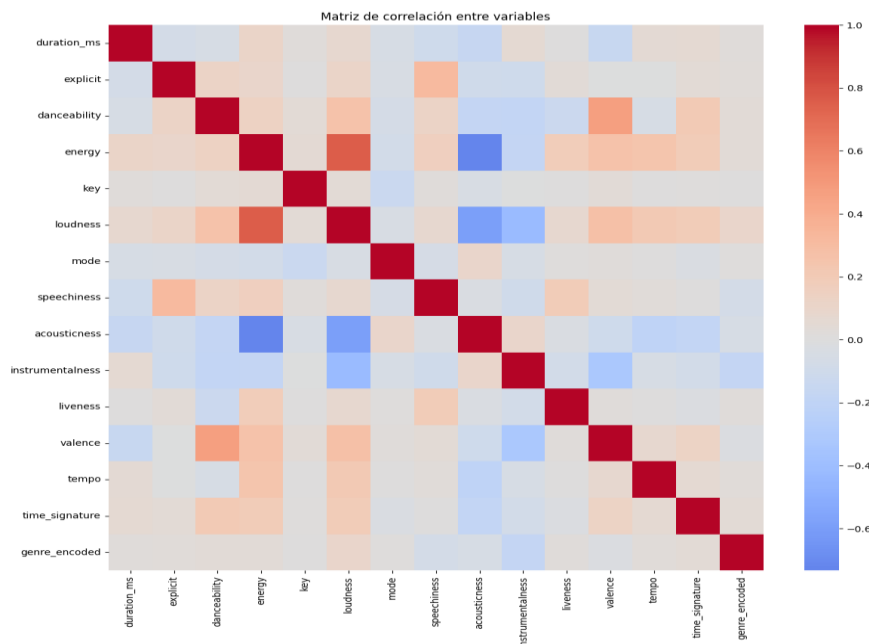
2.7 Distribuciones y transformaciones

Los histogramas mostraron sesgo pronunciado a la derecha en duration_ms, speechiness e instrumentalness. Se evaluó la transformación log1p en los modelos preliminares con Random Forest. En el modelo final CatBoost esta transformación no fue necesaria, ya que los árboles son invariantes a transformaciones monotónicas. La variable time_signature concentra más del 95% de sus valores en 4 (compás de 4/4), por lo que en algunas pruebas de modelos se deja por fuera del set de datos.



2.8 Correlación y multicolinealidad

Se identificaron dos pares con correlación superior a 0.7: energy–loudness ($r = 0.762$) y energy–acousticness ($r = 0.734$). En modelos de árboles la multicolinealidad no afecta la estabilidad ni el desempeño, por lo que se conservaron todas las variables en esta etapa, postergando la decisión al análisis de feature importance.



2.9 Análisis de variables — F-test

Se aplicó `f_regression` sobre el set de entrenamiento donde `genre_encoded` resultó la variable más relevante ($F_{\text{score}} = 21.560$), seguida de `instrumentalness`, `loudness`, `speechiness` y `explicit`. Las variables `energy` y `key` no mostraron relación lineal significativa ($p > 0.1$). La decisión final de eliminación se pospone al análisis de feature importance del modelo entrenado.

Variable	F_score	p_value
instrumentalness	586.400353	5.823027e-129
loudness	169.845445	8.963429e-39
speechiness	131.157379	2.449015e-30
explicit	121.177260	3.704772e-28
valence	110.587491	7.649384e-26
danceability	85.267534	2.683053e-20
acousticness	46.955123	7.328710e-12
tempo	14.230185	1.619009e-04
mode	12.609270	3.841145e-04
liveness	6.178191	1.293599e-02
duration_ms	5.610699	1.785410e-02
energy	0.216289	6.418835e-01
key	0.163422	6.860270e-01

Variables originales: 13
Variables seleccionadas: 11

3. Calibración del Modelo

3.1 Modelos base evaluados

Antes de la calibración se entrenaron varios modelos para establecer referencias de desempeño:

Modelo	RMSE
Árbol de decisión simple (max_depth=5)	23.0
Votación ponderada (lr, dt, nb, kn)	21.5
BaggingRegressor (DecisionTree, 10 est.)	20.8
RandomForest base (100 estimadores)	16.04

3.2 Feature importance y eliminación de variables

Tras entrenar el RandomForest base, se analizó la importancia de cada variable y se comparó con el F-test. La variable key fue la única señalada por ambos métodos como de baja contribución ($p=0.69$ en F-test, 2.9% de importancia), por lo que se eliminó. Las demás variables se conservaron.

	Variable	F_score	p_value	Importancia_RF
<pre># Umbral de corte umbral_fscore = 0.1 umbral_importancia = 0.03 # F-score (ya calculado antes) f_table_idx = f_table.set_index('Variable') # Feature importance RF importancias_idx = importancias.set_index('Variable')</pre>	acousticness	46.955123	7.328710e-12	0.104204
	danceability	85.267534	2.683053e-20	0.100222
	duration_ms	5.610699	1.785410e-02	0.098942
	valence	110.587491	7.649384e-26	0.098911
	tempo	14.230185	1.619009e-04	0.098800
	speechiness	131.157379	2.449015e-30	0.098407
	loudness	169.845445	8.963429e-39	0.094354
	energy	0.216289	6.418835e-01	0.088169
	liveness	6.178191	1.293599e-02	0.087351
	instrumentalness	586.400353	5.823027e-129	0.071604
	key	0.163422	6.860270e-01	0.042462
	mode	12.609270	3.841145e-04	0.009664
	explicit	121.177260	3.704772e-28	0.006912

3.3 Escalamiento

Se evaluaron StandardScaler y RobustScaler. Las diferencias de RMSE fueron inferiores a 0.002 puntos, confirmando que los árboles son invariantes a la escala. El escalamiento fue descartado como técnica de mejora.

```
scalers = {
    'standard': StandardScaler(),
    'robust': RobustScaler()
}

for scaler_name, scaler in scalers.items():
    # Escalar
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    # Entrenar RF
    rf = RandomForestRegressor(n_estimators=100, random_state=42, n_jobs=-1)
    rf.fit(X_train_scaled, y_train)

    # Evaluar
    y_pred = rf.predict(X_test_scaled)
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))
```

3.4 RandomizedSearchCV — calibración de hiperparámetros

Se calibró RandomForest con RandomizedSearchCV con los siguientes parámetros:

```
param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [None, 10, 20, 30],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'max_features': ['sqrt', 'log2', None]
}

search = RandomizedSearchCV(
    RandomForestRegressor(random_state=42, n_jobs=-1),
    param_distributions=param_grid,
    n_iter=30,
    cv=5,
    scoring='neg_root_mean_squared_error',
    random_state=42,
    n_jobs=-1,
    verbose=1
)

search.fit(X_train, y_train)
```

Los parámetros encontrados se documentaron y el modelo se reentrenó explícitamente con ellos:

Análisis de mejores parámetros calibrados:

Parámetro	Valor	Efecto
n_estimators	300	Define cuántos árboles conforman el ensemble. A mayor número, menor varianza en las predicciones al promediar más modelos. Se probaron 500 árboles con una mejora de solo 0.001 puntos de RMSE, confirmando que el modelo converge en 300 y que agregar más árboles no justifica el costo computacional.
max_depth	30	Controla la profundidad máxima de cada árbol. Sin límite (None), el árbol crece hasta memorizar el set de entrenamiento, lo que genera sobreajuste. Con profundidad 30 se permite capturar interacciones complejas entre variables manteniendo capacidad de generalización sobre datos nuevos.
min_samples_split	2	Número mínimo de muestras que debe tener un nodo para ser dividido. Con valor 2, el árbol puede dividir nodos con muy pocas observaciones, lo que maximiza la granularidad de las decisiones. Este valor es adecuado para un dataset de ~64.000 registros donde los nodos hoja tienen suficiente respaldo estadístico.
min_samples_leaf	1	Número mínimo de muestras requeridas en cada hoja. Con valor 1 se permite que las hojas contengan una sola observación, lo que le da al árbol máxima flexibilidad para ajustarse a los datos. El efecto de sobreajuste individual se compensa con el promedio de los 300 árboles del ensemble (bagging).
max_features	log2	Número de variables consideradas en cada split, calculado como $\log_2(n_features) = 4$ variables por división. Al evaluar solo un subconjunto aleatorio de features en cada nodo, los árboles del ensemble se vuelven más diversos entre sí, ya que cada uno aprende a partir de distintas combinaciones de variables. Esta diversidad es lo que hace al Random Forest más robusto que un árbol individual.

4. Entrenamiento del Modelo

4.1 Evolución del RMSE

A continuación, se muestra la evolución en la mejora del RMSE en los diferentes modelos entrenados hasta conseguir el mejor resultado:

Modelo	RMSE
RandomForest base	16.04
RandomForest + genre_encoded	16.38
RandomForest calibrado	15.70
CatBoost 1000 iter.	8.9993
CatBoost 2000 iter.	8.8156
CatBoost 3000 iter.	8.7408
CatBoost OOF + K-Fold (solución final)	7.09875

4.2 Justificación de CatBoost

CatBoost supera a Random Forest al incorporar variables categóricas de texto (track_id, artists, album_name, track_name, track_genre) de forma nativa mediante Ordered Target Encoding interno, pasando de RMSE 15.63 a 8.74 — una reducción del 44%. El algoritmo aprende que ciertos artistas y álbumes tienen patrones de popularidad consistentes, información que Random Forest no puede capturar.

```
cols_drop = ['popularity', 'Unnamed_0']
X_full_cat = dataTraining.drop(columns=[c for c in cols_drop if c in dataTraining.columns])
y_full = dataTraining['popularity']

# Split
X_train_cat, X_test_cat, y_train_cat, y_test_cat = train_test_split(
    X_full_cat, y_full, test_size=0.2, random_state=42
)

# Identificar columnas categoricas
cat_features = ['track_genre', 'artists', 'track_id', 'album_name', 'track_name']

# Modelo
cb_model = CatBoostRegressor(
    iterations=3000,
    learning_rate=0.05,
    depth=8,
    random_seed=42,
    verbose=100
)

cb_model.fit(
    X_train_cat, y_train_cat,
    cat_features=cat_features,
    eval_set=(X_test_cat, y_test_cat)
)

y_pred_cat = cb_model.predict(X_test_cat)
rmse_cat = np.sqrt(mean_squared_error(y_test_cat, y_pred_cat))
```

5. Modelo Optimizado — Solución Final

Como solución final se implementó una versión más sofisticada de CatBoost con tres mejoras sobre el modelo exploratorio: OOF encoding, variables compuestas de texto y validación cruzada K-Fold para la predicción.

5.1 Feature engineering avanzado

Se agregaron variables derivadas del texto y llaves compuestas que permiten identificar combinaciones únicas de entidades:

```
# crear variables numericas complementarias
datos["energia_por_bailabilidad"] = datos["energy"] * datos["danceability"]
datos["valencia_por_bailabilidad"] = datos["valence"] * datos["danceability"]
datos["energia_por_loudness"] = datos["energy"] * datos["loudness"]
datos["acustica_por_energia"] = datos["acousticness"] * datos["energy"]
datos["tempo_por_bailabilidad"] = datos["tempo"] * datos["danceability"]
datos["speech_por_explicit"] = datos["speechiness"] * datos["explicit"]
datos["loudness_abs"] = datos["loudness"].abs()
datos["energia_sobre_loudness_abs"] = datos["energy"] / (datos["loudness_abs"] + 1)
datos["bailabilidad_energia_valencia"] = datos["danceability"] * datos["energy"] * datos["valence"]
datos["acustica_menos_energia"] = datos["acousticness"] - datos["energy"]

# variables compuestas
datos["llave_cancion_artista"] = datos["track_name"] + " | " + datos["artists"]
datos["llave_cancion_artista_album"] = datos["track_name"] + " | " + datos["artists"] + " | " + datos["album_name"]
datos["llave_artista_album"] = datos["artists"] + " | " + datos["album_name"]
datos["llave_artista_genero"] = datos["artists"] + " | " + datos["track_genre"]
datos["llave_album_genero"] = datos["album_name"] + " | " + datos["track_genre"]
```

5.2 Target Encoding Out-Of-Fold (OOF)

El OOF encoding calcula las estadísticas por categoría usando únicamente los folds de entrenamiento, eliminando data leakage. Se calculan media, tasa de ceros y tasa baja con suavizados de 5, 20 y 50 para 10 columnas de alta cardinalidad.

Es fundamental recalcar que, para garantizar la integridad del modelo y evitar el *data leakage* (fuga de datos), el proceso de preprocesamiento siguió estrictamente la lógica de ajuste y transformación requerida. Se realizó el cálculo de las estadísticas (`.fit_transform()`) únicamente sobre los datos de entrenamiento (o los folds correspondientes en el esquema OOF) y se aplicó la transformación (`.transform()`) sobre los datos de validación y prueba, utilizando exclusivamente los parámetros aprendidos en la fase de entrenamiento.

```
# encoding oof
def agregar_estadisticas_oof(train, test, columnas_grupo, objetivo, n_folds, semilla):
    train = train.copy()
    test = test.copy()

    media_global = train[objetivo].mean()
    tasa_cero_global = (train[objetivo] == 0).mean()
    tasa_baja_global = (train[objetivo] <= 5).mean()

    suavizados = [5, 20, 50]
    kfold = KFold(n_splits=n_folds, shuffle=True, random_state=semilla)

    for columna in columnas_grupo:
        print("Codificando:", columna)

        for suavizado in suavizados:
            train[f"{columna}_media_s{suavizado}"] = np.nan
            train[f"{columna}_cero_s{suavizado}"] = np.nan
            train[f"{columna}_baja_s{suavizado}"] = np.nan

        train[f"{columna}_conteo"] = np.nan

        for idx_train, idx_validacion in kfold.split(train):
            datos_entrenamiento = train.iloc[idx_train]
            datos_validacion = train.iloc[idx_validacion]

            estadisticas = datos_entrenamiento.groupby(columna)[objetivo].agg(
                media="mean",
                conteo="count",
                tasa_cero=lambda x: (x == 0).mean(),
                tasa_baja=lambda x: (x <= 5).mean()
            )
```

5.3 Entrenamiento K-Fold y parámetros finales

Se entrenaron 5 modelos CatBoost (uno por fold) y se promediaron las predicciones sobre el test para reducir varianza:

```
# k-fold - prediccion
SEMILLA = 2025
N_FOLDS = 5
kfold = KFold(n_splits=N_FOLDS, shuffle=True, random_state=SEMILLA)
predicciones_oof = np.zeros(len(train))
predicciones_test = np.zeros(len(test))
puntajes = []
for fold, (idx_train, idx_validacion) in enumerate(kfold.split(train)):
    print("-" * 80)
    print(f"Fold {fold}")
    X_train = train.iloc[idx_train][variables_modelo]
    y_train = train.iloc[idx_train][OBJETIVO]
    X_validacion = train.iloc[idx_validacion][variables_modelo]
    y_validacion = train.iloc[idx_validacion][OBJETIVO]
    pool_train = Pool(X_train, y_train, cat_features=columnas_categoricas)
    pool_validacion = Pool(X_validacion, y_validacion, cat_features=columnas_categoricas)
    modelo = CatBoostRegressor(**parametros_catboost)
    modelo.fit(
        pool_train,
        eval_set=(pool_validacion, y_validacion),
        use_best_model=True
    )
    pred_validacion = np.clip(modelo.predict(X_validacion), 0, 100)
    pred_fold_test = np.clip(modelo.predict(test[variables_modelo]), 0, 100)

    predicciones_oof[idx_validacion] = pred_validacion
    predicciones_test += pred_fold_test / N_FOLDS

rmse_fold = calcular_rmse(y_validacion, pred_validacion)
puntajes.append(rmse_fold)
```

El modelo final se configuró mediante un proceso de refinamiento técnico, seleccionando 6,000 iteraciones y un `learning_rate` reducido de 0.022. Esta combinación permite que el modelo aprenda patrones muy sutiles en los 125 géneros musicales sin converger prematuramente. Se utilizó una profundidad (`depth`) de 8 para capturar interacciones complejas entre las características de audio y el texto, junto con una regularización `l2_leaf_reg` de 10 y un `random_strength` de 2.0 para controlar el sobreajuste. Estos parámetros fueron fundamentales para estabilizar el RMSE y asegurar que el modelo generalice correctamente ante la alta variabilidad de los datos de Spotify.


```
# parametros del modelo final
parametros_catboost = {
    "loss_function": "RMSE",
    "eval_metric": "RMSE",
    "iterations": 6000,
    "learning_rate": 0.022,
    "depth": 8,
    "l2_leaf_reg": 10,
    "random_strength": 2.0,
    "bagging_temperature": 1.0,
    "random_seed": SEMILLA,
    "verbose": 500,
    "allow_writing_files": False,
    "thread_count": -1
}
```

5.4 Resultado final optimizado

A continuación, se muestra la comparación del resultado del modelo optimizado contra un CatBoost con 3000 mil iteraciones:

Modelo	RMSE
CatBoost exploratorio (3000 iter.)	8.74
CatBoost OOF + K-Fold (solución final) (6000 iter.)	7.09875

6. Disponibilización — API en la Nube

6.1 Exportación del modelo

El modelo final se entrenó utilizando todo el conjunto de entrenamiento disponible y posteriormente se exportó en formato .cbm, propio de CatBoost. Este archivo contiene la estructura del modelo entrenado y los parámetros necesarios para realizar predicciones. Adicionalmente, se generó un archivo de metadata en formato .pkl, el cual almacena la información necesaria para reproducir el preprocesamiento en producción. En particular, este archivo contiene entre sus datos, lista de variables usadas por el modelo columnas categoricas usadas por CatBoost, columnas usadas para target encoding, valores globales del target, estadísticas por grupo para los encodings, parametros de suavizado usados en los encodings. Esto permite que la API pueda recibir una nueva canción, reconstruir las mismas variables usadas durante el entrenamiento y generar una predicción consistente con el modelo original.

6.2 Cargar el modelo y disponibilizar el modelo en API

Para disponibilizar el modelo se creó una instancia EC2 en AWS. Posteriormente, se estableció conexión con la instancia mediante SSH y se cargó la carpeta del proyecto, la cual contiene los archivos necesarios para ejecutar el servicio.

La API fue desarrollada usando FastAPI, ya que permite construir servicios de predicción de forma sencilla, estructurada y eficiente. Dentro de la API se carga el modelo CatBoost exportado y el archivo de metadata, de modo que cada solicitud pueda ser procesada con el mismo flujo aplicado durante el entrenamiento. El servicio fue contenerizado usando Docker, con el objetivo de estandarizar el ambiente de ejecución y evitar problemas asociados a dependencias, versiones de librerías o configuraciones particulares de la máquina. De esta forma, la API puede ejecutarse de manera consistente tanto en local como en la instancia EC2.


```

unpacking containerio [2.2.1-ubuntu17.24.04.2] ...
selecting previously unselected package dns-root-data.
preparing to unpack .../4-dns-root-...
unpacking dns-root-data (20240718)
selecting previously unselected package ML_API_SPOTIFY.
preparing to unpack .../5-ML_API_SPOTIFY -- --zsh -- 140x30
unpacking dnsmasq-base (2.9
selecting previously unselected package Diagrama_UML_SpotiAlpes.pdf
preparing to unpack .../6-d
unpacking docker.io (29.1.3
selecting previously unselected package Documentos Universidad
preparing to unpack .../7-u
unpacking ubuntu-fan (0.12
setting up dnsmasq-base (2
setting up runo (1.3.4-0ubu
setting up dns-root-data (2
setting up bridge-utils (1
setting up pigz (2.8-1) ...
setting up containers (2.2
Created symlink /etc/system
setting up ubuntu-fan (0.12
Created symlink /etc/system
setting up docker.io (29.1
info: Selecting CID from re
info: Adding group 'docker'
Created symlink /etc/system
Created symlink /etc/system
Processing triggers for dbu
Processing triggers for man
Scanning processes...
Scanning linux images...
Running kernel seems to be
no services need to be rest
no containers need to be re
no user sessions are runnin
no VM guests are running on
ubuntu@ip-172-31-33-148:~$
docker version 29.1.3, build
ubuntu@ip-172-31-33-148:~$
ML_API_SPOTIFY$
ubuntu@ip-172-31-33-148:~$ cd ML_API_SPOTIFY/
ubuntu@ip-172-31-33-148:~/ML_API_SPOTIFY$ ls
Dockerfile  app.py  metadata_spotify.pkl  modelo_catboost_spotify.cbm  requirements.txt  test_main.http
ubuntu@ip-172-31-33-148:~/ML_API_SPOTIFY$
--> Removed intermediate container 6a7aa3f2801b
--> b8ee8fb09dfd
Step 5/9 : COPY app.py .
--> 10ebf0258fb6
Step 6/9 : COPY modelo_catboost_spotify.cbm .
--> 81f0dc4b087e
Step 7/9 : COPY metadata_spotify.pkl .
--> bf9f916a5336
Step 8/9 : EXPOSE 8000
--> Running in 8b67aa701113
--> Removed intermediate container 8b67aa701113
--> 3fc87498c920
Step 9/9 : CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
--> Running in 04e22817cadf
--> Removed intermediate container 04e22817cadf
--> e277ba66a4d
Successfully built e277ba66a4d
Successfully tagged spotify-popularity-api:latest
ubuntu@ip-172-31-33-148:~/ML_API_SPOTIFY$

```

6.3 Consumo de API Spotify

La API expone un endpoint de predicción mediante el método POST. Este endpoint recibe como entrada un objeto en formato JSON con la información de una canción, aplica internamente el preprocesamiento completo y retorna la popularidad estimada por el modelo.

los datos para poder consumir el API son los siguientes:

URL: <http://18.191.169.202:8000/predict>

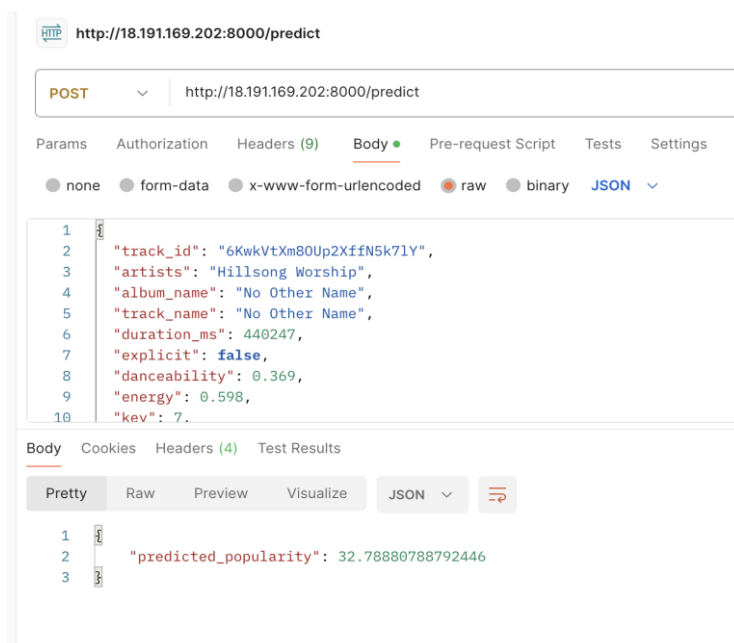
Headers: Content-Type: application/json

BODY TEST:

```

{
  "track_id": "6KwkVtXm8OUp2XffN5k7IY",
  "artists": "Hillsong Worship",
  "album_name": "No Other Name",
  "track_name": "No Other Name",
  "duration_ms": 440247,
  "explicit": false,
  "danceability": 0.369,
  "energy": 0.598,
  "key": 7,
  "loudness": -6.984,
  "mode": 1,
  "speechiness": 0.0304,
  "acousticness": 0.00511,
  "instrumentalness": 0.0,
  "liveness": 0.176,
  "valence": 0.0466,
  "tempo": 148.014,
  "time_signature": 4,
  "track_genre": "world-music"
}

```



6.4 Predicciones de prueba sobre el set de validación

Se procedió a validar la API desplegada en AWS EC2 realizando peticiones al endpoint /predict. Para demostrar la robustez de la solución, se evaluaron múltiples observaciones del set de validación. Los resultados obtenidos confirman que la API responde correctamente a las solicitudes JSON y devuelve predicciones de popularidad coherentes con el entrenamiento del modelo. A continuación se presentan dos ejemplos de estas pruebas:

Observación	Características	Predicción
The Weeknd — pop	energy=0.73, dance=0.514, loudness=-5.9	87.3
Susumu Yokota — idm	energy=0.509, dance=0.583, loudness=-9.7	12.1

7. Conclusiones

El proyecto evidenció que la popularidad en Spotify depende más de la identidad de la canción (artista, álbum, género) que de sus características de audio. El proceso exploratorio con Random Forest estableció una línea base sólida (RMSE 16.04), confirmó que el escalamiento no aporta en árboles, y permitió calibrar hiperparámetros que redujeron el error a 15.70. La incorporación de feature engineering bajó el RMSE a 15.63.

El salto más significativo se dio al pasar a CatBoost habilitando las variables de texto: de 15.63 a 8.74, una reducción del 44%. El Ordered Target Encoding interno de CatBoost captura de forma más precisa los patrones de popularidad por artista y álbum que el target encoding simple de media.

Hallazgos clave:

- Variables de texto (artists, album_name, track_id) tienen mayor poder predictivo que las características de audio.
- CatBoost con encoding nativo supera al Random Forest en 44% de reducción de RMSE.
- OOF encoding elimina data leakage y mejora la calidad del encoding vs media simple.
- K-Fold para predicción final estabiliza el resultado vs un único split train/test.

La integración de hiperparámetros robustos (6,000 iteraciones y regularización L2) junto con una arquitectura de despliegue en la nube mediante FastAPI y Docker, permitió alcanzar un equilibrio óptimo entre precisión técnica y disponibilidad operativa, cumpliendo con todos los objetivos planteados para la predicción de popularidad en Spotify.